本章路線

這章要回答的核心問題是：作業系統如何把連續的 logical address 安全映射到有限且分散的 physical memory？

前置知識。 二進位位址、暫存器與 memory bus、process 隔離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 推導 base/limit、segment、page 的轉換
- 比較 internal/external fragmentation
- 分析 TLB 命中率與 page table 層級

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1 Background

- CPU 產生的 logical address 經 memory-management unit 轉成 physical address。process 之間不能直接使用彼此的實體位址。
- base register 加 offset 提供基本重定位，limit register 限制合法偏移。硬體必須在每次存取檢查，不能信任使用者先自行判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1　模型、時序與反例：Background

模型假設。 CPU 產生 logical address，MMU 在每次存取前執行 base/limit 或 page 保護；位址欄位寬度固定。

不變量與時序。 offset 必須小於 segment limit；k-bit page offset 的數值必須小於 2^k，不能拿數值與位元寬度直接比較；未通過檢查的位址不得到達實體記憶體。

反例或邊界。 使用者程式自行檢查指標範圍後，核心直接信任該指標，惡意程式可繞過檢查。

完整算例。 base=0x8000、limit=0x2000，offset=0x1FFF 合法，0x2000 越界。

思考題。 protection check 放在軟體與硬體分別對延遲和信任有何影響？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.2 Swapping

- swapping 把整個 process 移到 backing store，以釋放主記憶體；需要可容納 process 的連續或可映射空間與恢復狀態。
- 若每個 page 都可選擇性載入，demand paging 比 whole-process swapping 細緻；成本與政策不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.2　模型、時序與反例：Swapping

模型假設。 backing store 容量足以保存可換出的狀態，swap manager 能保存 process 的完整位址/資源 metadata。

不變量與時序。 換出與換入期間，同一 process 不得有兩份被誤認為最新狀態；使用中的 I/O buffer 不得被換出。

反例或邊界。 DMA 尚未完成的 page 被換出並回收 frame，裝置會寫到新的使用者資料。

完整算例。 P1 80 KiB、backing store 有 100 KiB 空位，整 process 換出；恢復前標記所有相關 TLB entries 失效。

思考題。 demand paging 與 whole-process swap 的主要成本和控制點有何不同？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.3 Contiguous allocation

- 每個 process 佔一段連續 frames。first-fit、best-fit、worst-fit 依 free list 選擇；都只處理當下，不能保證未來需求。
- 配置與釋放形成 external fragmentation，可用 compaction 移動 process，但需停止或協調位址相關活動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.3　模型、時序與反例：Contiguous allocation

模型假設。 free list 只包含可配置 holes，配置與釋放由核心序列化；process size 不變或最大需求已知。

不變量與時序。 各配置區段不重疊，free holes 與 allocated ranges 的總和被 resource model 解釋；遷移時所有位址引用更新。

反例或邊界。 總空閒 20 KiB 但被切成 4 個 5 KiB holes，仍無法配置 10 KiB 的連續需求。

完整算例。 依此 free-list 順序有 holes 100、500、200 KiB，配置 180 KiB：first-fit 跳過 100 後用 500，剩 320；best-fit 選 200，剩 20；worst-fit 選 500，剩 320。

思考題。 何種 workload 會讓 best-fit 留下大量難用的小洞？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.4 Segmentation

- logical address 分成 segment number 與 offset；segment table entry 有 base 與 limit。偏移超出 limit 觸發 fault。
- segmentation 反映程式邏輯區段，可見 size 變動與 external fragmentation；paging 固定大小，消除 external fragmentation 但產生 page 內部浪費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4　模型、時序與反例：Segmentation

模型假設。 segment 有 base、limit，硬體在 offset 超界時產生 fault；segment 的生命週期由核心追蹤。

不變量與時序。 合法 offset 必須小於 limit，轉換後的 range 不與其他 process 的受保護 segment 重疊。

反例或邊界。 只檢查 base 而不檢查 limit，大 offset 可讀到相鄰段。

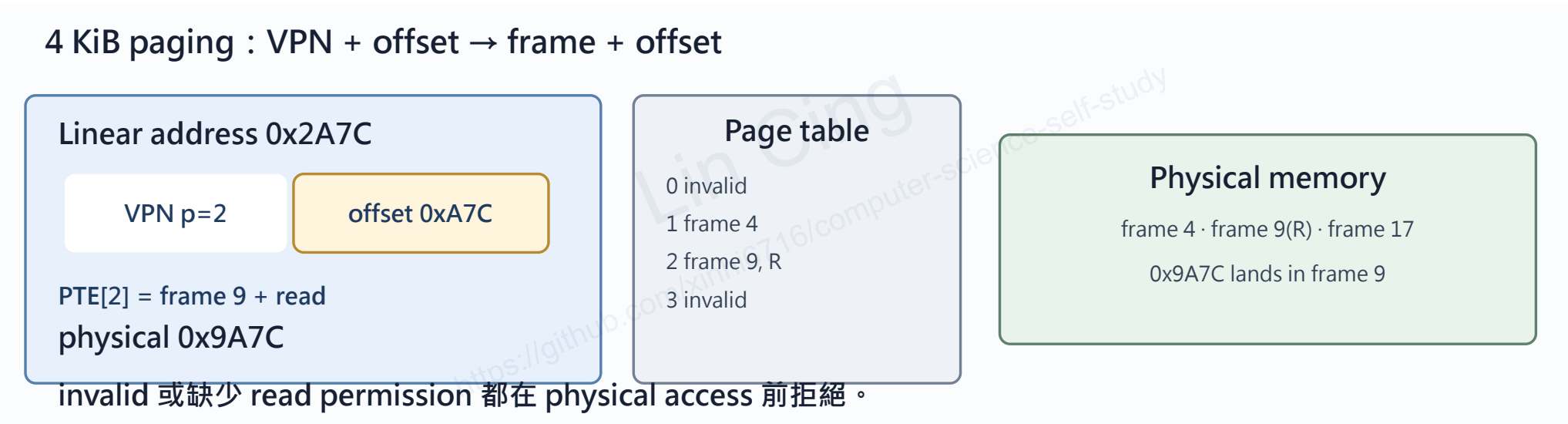
完整算例。 segment 2 base=0x40000、limit=0x3000，offset 0x2FFF 合法，0x3000 fault。

思考題。 為何 segmentation 能表達程式結構，卻仍需 paging 補足配置？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5 Paging

- logical address 分成 page number p 與 offset d。page table[p] 給 frame number f，實體位址為 f×page_size+d。
- page 大小是 trade-off。較小頁減少 internal fragmentation，但增大 table 與轉換 overhead；較大頁降低項目數，卻可能浪費每個物件的尾端空間。



8.5.1 Fragmentation 分類

- internal fragmentation 是配置單位大於需求，未使用空間位於配置區塊內部；fixed pages 與 blocks 都會造成。
- external fragmentation 是總空閒足夠但分散，沒有單一連續區塊滿足請求；contiguous allocation 與 variable segments 會遇到。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5.2 TLB 與有效存取時間

- TLB 快取 page table entry；命中可用一次 TLB lookup 加 memory access，miss 還要存取 page table。
- 若命中率 h、TLB access t、memory access m、miss 需 k 次 page-table access，簡化 EMAT = h(t+m)+(1-h)(t+k m+m)，硬體平行查找時 t 可依架構移除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5.3 保護與共享

- 每個 page table entry 可含 valid、read/write、execute、user/kernel 等位元；另一 process 只有在其 page table 映射同一 frame 時才可共享。
- 共享唯讀 code 可節省記憶體，但寫入或可執行權限若設錯，就會破壞隔離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5　模型、時序與反例：Paging

模型假設。 page size 是 2^k，entry 有 frame number、valid 與權限；TLB 一致性由硬體或軟體維護。

不變量與時序。 只有 valid 且權限允許的 page 可完成轉換；page table 更新後，舊 TLB translation 不得再被使用。

反例或邊界。 核心解除映射但不 flush TLB，CPU 仍可能以舊 entry 讀取已配置給其他 process 的 frame。

完整算例。 4 KiB page、frame 9、offset 0xA7C，實體位址 0x9000+0xA7C。

思考題。 哪一個操作既改 page table 又要求 cache/TLB 維護？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.6 Page table 結構

- 32 位元與 64 位元位址造成 page table 太大，因此使用 hierarchical、hashed 或 inverted table。
- 多層 table 省下大部分未使用的下層，但每次 TLB miss 可能增加記憶體存取層級。64-bit 位址只使用架構規範的部分位元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.6　模型、時序與反例：Page table 結構

模型假設。 hierarchical table 每一層 entry 可指向下層或標示不存在；位址切分與架構模式一致。

不變量與時序。 沿 root 到下層的索引必須完整且有效；任何層缺失都不能到達實體 frame。

反例或邊界。 64-bit 位元全用來建多層表會造成荒謬大小，必須只用架構支援範圍或 sparse layout。

完整算例。 32-bit linear address 0x12345678、4 KiB page：p1=0x048、p2=0x345、d=0x678。若 outer[0x048] 指向 inner table T、T[0x345] 指向 frame 0x0ABC，則 PA=0xABC678；若 inner entry 的 present=0，立即 fault，不以 VA 本身充當 PA。

思考題。 多層 table 減少的項目數會換來哪些 miss 與存取成本？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.7 Intel 32 與 64 位元架構

- IA-32 固定 protected mode、4 KiB pages、linear address 0x12345678。32-bit 線性位址切成 10/10/12；分割結果為 PD index 0x048、PT index 0x345、offset 0x678。
- 若 page-directory base=0x5000、PT base=0x6000，PDE 在 0x5000+4×0x048=0x5120，PTE 在 0x6000+4×0x345=0x6D14。有效 entries 0x00006007 與 0x00ABC007 給 frame 0xABC，PA=0xABC678。
- x86-64 固定 4-level、4 KiB pages、48-bit canonical VA。0x00007FFF_FFFFF123 的 indices 為 0x0FF/0x1FF/0x1FF/0x1FF、offset 0x123；canonical 檢查要求 bit47 複製到 bits63:48。
- 固化 CR3=0x1000、PDPT=0x2000、PD=0x3000、PT=0x4000。四 entries 0x2007/0x3007/0x4007/0xABC007 分別位於 0x17F8/0x2FF8/0x3FF8/0x4FF8；最後 PA=0xABC123。
- 0x00008000_00000000 的 bits63:48=0、bit47=1，在任何 table lookup 前被拒。present、user、read/write 與 NX 是獨立後續判斷；缺 entry、唯讀 PTE 與 NX 都可令合法 canonical VA fault。

8.7　模型、時序與反例：Intel 32 與 64 位元架構

模型假設。 IA-32 輸入是已通過 segment limit 的 32-bit linear address；其 PD/PT entries 各 4 bytes。x86-64 輸入是 48-bit canonical range，四層 entries 各 8 bytes、每表 512 entries；CR3 指向 present PML4，且 user access 沿路各層允許。

不變量與時序。 table entry address=table base+index×entry_size；下一層 base 來自 present entry 的 page-aligned address。index 只選 entry，frame×0x1000+offset 才給 physical address。canonical 先行，present 與 effective read/write/user/NX 再由各層 flags 合取或檢查。

反例或邊界。 0x00008000_00000000 是 noncanonical；0x00007FFF_FFFFF000 即使 canonical，若 PML4/PDPT/PD/PT 任一 present=0 仍 page fault；同一路徑若 PTE RW=0，read 可成功而 write 必須 protection fault。

完整算例。 x86-64 0x00007FFF_FFFFF123→indices 0x0FF/0x1FF/0x1FF/0x1FF、offset 0x123，entry offsets 0x7F8/0xFF8/0xFF8/0xFF8，PA=0xABC123。IA-32 0x12345678→PDE@0x5120、PTE@0x6D14、offset 0x678，PA=0xABC678。

思考題。 為何檢查出 canonical=true 後仍不能宣稱一次 user read/write 必定成功？

8.7　IA-32 protected mode：從 linear address 到 page entry

本模型的 IA-32 輸入是已通過 segment descriptor 檢查的 32-bit linear address；page-size 固定 4 KiB。segment selector 先用 LDT/GDT descriptor 的 base 與 limit 形成 linear address，paging unit 再把 linear address 轉成 physical address。因此 `0x12345678` 是 linear address，不把它誤稱為尚未經過 segmentation 的 logical address。

每張 IA-32 page directory 與 page table 有 $4096/4 = 1024$ entries，索引各 10 bits。令 linear address 為 L：

$$\mathrm{PDI} = (L \gg 22)\&0x3FF, \quad \mathrm{PTI} = (L \gg 12)\&0x3FF, \quad \mathrm{offset} = L\&0xFFF.$$

固定 CR3 page-directory base=`0x5000`、下一層 page-table base=`0x6000`。entry address 由 base+index×4 得到：

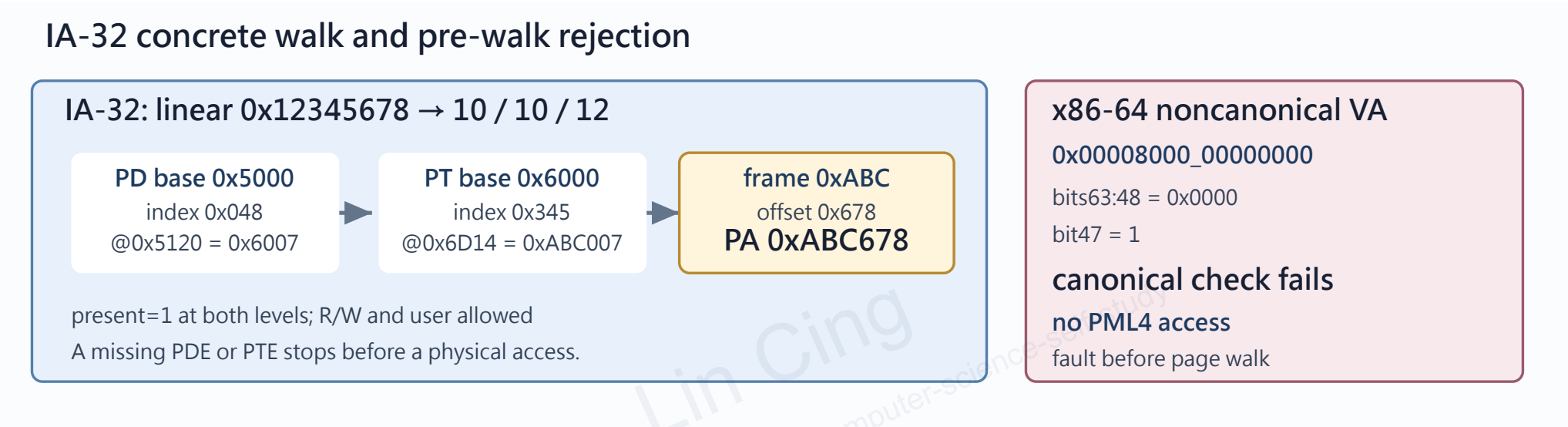
層級	index	entry 位址	表項內容	解讀
page directory	`0x048`	`0x5000+4×0x048=0x5120`	`0x00006007`	present；next PT base=`0x6000`；R/W、user 允許
page table	`0x345`	`0x6000+4×0x345=0x6D14`	`0x00ABC007`	present；4 KiB frame=`0xABC`；R/W、user 允許

8.7 IA-32 protected mode：從 linear address 到 page entry （續）

因此 offset=`0x678`，physical address=`0xABC000+0x678=0xABC678`。這一路徑使用 page directory、page table 與目標 page 三個 4 KiB 記憶體頁；table 本身的 page-table 配置成本另計，但由上表的 entries 可完全重建。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.7 IA-32 protected mode：從 linear address 到 page entry （續）



若 PDE 的 present=0，paging unit 在讀下層 PT 前就 fault；若 PTE 的 present=0，不能把 frame 欄位解成有效記憶體。若 entry 指向 4 MiB page，PDE 的 Page Size flag 會讓 PDI 之後的 22 bits 成為 offset，直接跳過 page table；這個 encoding 不在本頁固定的 4 KiB trace 中。

8.7　x86-64 canonical address 與四層頁表

OSC 9e 的 x86-64 案例使用 48-bit virtual address 與四層 4 KiB paging。有效位元為 `[47:39]`、`[38:30]`、`[29:21]`、`[20:12]`，每層 9-bit index；offset 為 `[11:0]`。每張 table 有 $4096/8 = 512$ entries。

本頁固定48-bit canonical模式，輸入先限定0≤v<2^64。Canonical condition要求bit47複製到所有`[63:48]`，不能只比較bit63與bit47：

$$\text{canonical}(v) = \left((v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases} \right)$$

反例`v=0x0001_0000_0000_0000=2^48`的bit63=bit47=0，但bits63:48=1而非0，故noncanonical，不能walk。合法lower-half位址若單獨把任一bit48…63翻成1，都破壞sign extension；合法upper-half若把任一對應bit翻成0亦然。四個邊界為0與0x00007FFFFFFFFFFF合法、0x0008000000000000至0xFFFF7FFFFFFFFFFF非法、0xFFFF800000000000至0xFFFFFFFFFFFFFFFF合法。

固定 CR3=`0x1000`，沿路 table bases 為 PDPT=`0x2000`、PD=`0x3000`、PT=`0x4000`。目標 VA=`0x00007FFF_FFFFF123` 的 bit47=0、bits63:48=0，所以 canonical。各層都提 9 bits：

8.7 x86-64 canonical address 與四層頁表 (續)

層級	index	entry offset	entry 位址	entry 值	下一個 frame/table
PML4	`0x0FF`	`0x7F8`	`0x17F8`	`0x2007`	PDPT base `0x2000`
PDPT	`0x1FF`	`0xFF8`	`0x2FF8`	`0x3007`	PD base `0x3000`
PD	`0x1FF`	`0xFF8`	`0x3FF8`	`0x4007`	PT base `0x4000`
PT	`0x1FF`	`0xFF8`	`0x4FF8`	`0xABC007`	data frame `0xABC`

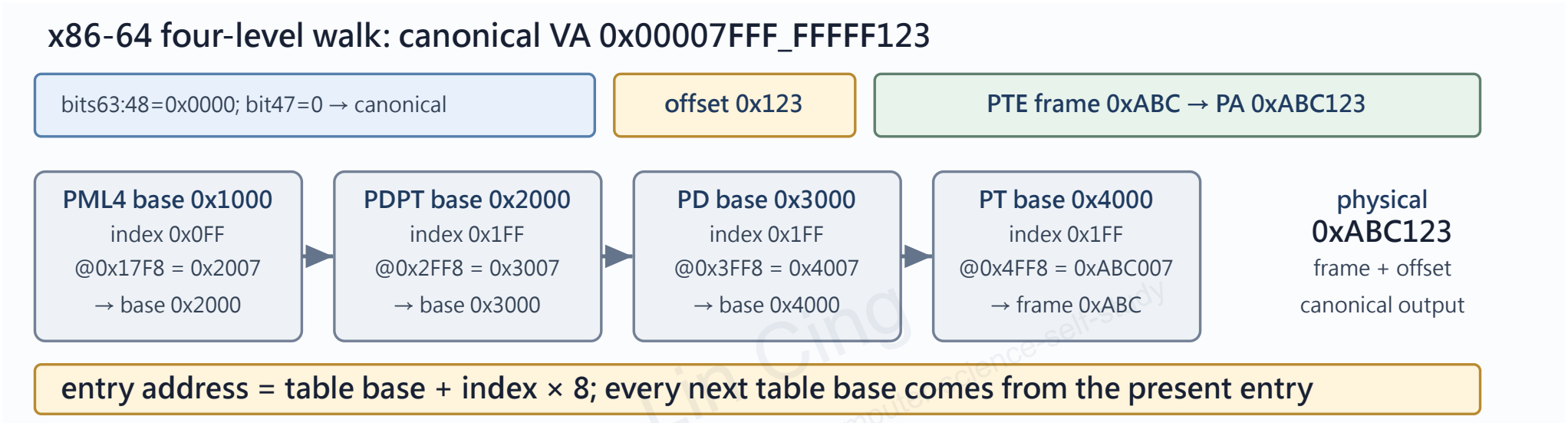
entry$_\ell$ 位址的一般式是

$$address_\ell = base_\ell + index_\ell \times 8.$$

最後 offset=`0x123`，故 physical address=`0xABC000+0x123=0xABC123`。若只算 indices，仍不足以重建表格；本例另給每層 base、entry 位址與表項值，任何一步都可從上表重新展開。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.7 x86-64 canonical address 與四層頁表 (續)



2 MiB 與 1 GiB large-page encodings 會在較上層終止 walk，並把較多低位址 bits 當 offset。它們需要不同 entry format 與 alignment；本頁的 9/9/9/9/12 只描述指定的 4 KiB 路徑。

8.7　合法、非法與權限 fault 的完整分流

canonical、present、U/S、R/W 與 NX 回答不同問題。canonical 失敗時不產生 page-table memory access；canonical 通過後，page walk 先找 mapping，再依操作合併權限。

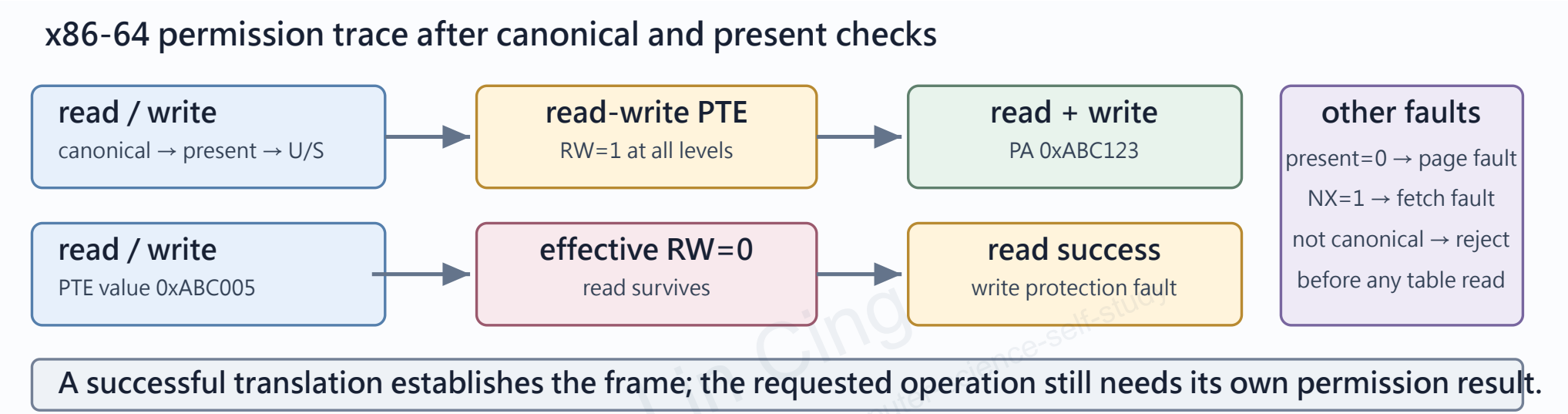
案例	canonical	mapping	effective permission	read	write	execute
本頁正常路徑	true	四層 present	全層 U/S=1、R/W=1、NX=0	`0xABC123`	`0xABC123`	允許
PTE 改為 `0xABC005`	true	四層 present	全層 U/S=1；R/W=0；NX=0	`0xABC123`	protection fault	允許
本次PML4[0x0FF] absent	true	present=0	不成為有效映射	page fault	page fault	instruction page fault
PTE 加上 NX	true	四層 present	R/W 允許；NX=1	資料讀取成功	資料寫入成功	instruction fetch fault
`0x00008000_00000000`	false	不應 walk	無法定義	reject	reject	reject

8.7　合法、非法與權限 fault 的完整分流 （續）

canonical check 不可取代 U/S；user 存取要求沿途 entry 的 user 權限允許。write 需要 effective R/W 條件；NX 由資料讀取與 instruction fetch 的操作類型區分。root 高位、table entry 與 page frame 都存在，也不等於任意操作有權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.7　合法、非法與權限 fault 的完整分流 （續）



這個模型固定 48-bit 4-level 設定。PAE、large pages、5-level paging 或不同 physical-width 會改變欄位與 entry 數，必須換成對應模式；不能把本頁的 masks 當成所有 Intel CPU 的通則。TLB 有效期則由 entry 更新與必要的 invalidation 決定。

8.8 ARM 架構

- 本節固定 ARMv7-A、AArch32、short-descriptor、TTBCR.N=0、SCTLR.AFE=0 與 4 KiB small page。N=0 時所有 VA 使用 TTBR0，且 root 是 4096×4 bytes=16 KiB 的完整 L1 table；VA[31:20] 選 entry。
- P1 root=0x8000 佔 0x8000–0xBFFF；P2 root=0xC000 佔 0xC000–0xFFFF。P1 VA 0x00101000 的 L1 entry@0x8004，P2 VA 0x00202000 的 entry@0xC008。roots 16 KiB 對齊且不重疊。
- coarse L1 指 1 KiB 的 L2 table，只需 1 KiB 對齊。P1 L2=0x10000、P2 L2=0x10400，L2 entry 分別 @0x10004 與@0x10408；兩個 1 KiB tables 也不與 root 或其他 table 重疊。
- L1 raw words為0x10061/0x10461。L2 raw word為0x40862：frame0x40000、type=small page、XN=0、nG=1；small-page AP[2]在bit9、AP[1:0]在bits[5:4]，故AP=010。TEX=001且C=B=0在SCTLR.TRE=0下解為Normal non-cacheable。
- 只有在DACR D3=Client（01）時AP才受檢查；AFE=0的AP=010使PL1可讀寫、PL0 read-only，因此read成功、write permission fault。D3=Manager不檢查AP；舊raw0x40802實際AP=000，正確decode後連PL0 read都fault。

8.8　模型、時序與反例：ARM 架構

模型假設。 ARMv7-A AArch32 short-descriptor；TTBCR.N=0、TTBR1停用、TTBR0 roots為完整16 KiB；SCTLR.AFE=0、SCTLR.TRE=0、WXN=UWXN=0；DACR D3=Client；P1/P2 L2 tables各1 KiB；PL1建表、PL0存取、TLB先無命中。

不變量與時序。 TTBR0 base 16KiB 對齊，root range不得與任何 L2 storage重疊；coarse table base bits[31:10]給1KiB對齊。L1 entry address=root+index×4；L2 entry address=L2 base+index×4；只有有效 translation 且 domain/client 允許後才檢查 AP。

反例或邊界。 宣告16 KiB root卻只配置4 KiB會越界；P2 root0x9000既未16KiB對齊、也與 P1 root0x8000–0xBFFF重疊；P1 L2=0xA000或P2 L2=0xB000都落在P1 root。即使entry address看似算得出來，也不能執行 walk。

完整算例。 P1 00101000→L1[001]@8004→L2@10000→[001]@10004；P2 00202000→L1[002]@C008→L2@10400→[002]@10408。兩者frame=40000，L2 word=0x40862：AP2=0、AP1=1、AP0=0，TEX/C/B=001/0/0；D3=Client時PL0 read成功、write permission fault。舊0x40802正確decode成AP000而拒PL0 read。

8.8　模型、時序與反例：ARM 架構 （續）

思考題。 把 DACR D3由 Client改為 Manager、或把 SCTLR.AFE由0改為1後，為何不能沿用原本的 AP 判定與 fault 結論？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.8　ARMv7-A AArch32：short-descriptor 的兩層 walk

本頁固定 ARMv7-A、AArch32、short-descriptor translation format、`TTBCR.N=0`、`SCTLR.AFE=0`、PL1 建表及 4 KiB small pages。這段不是 AArch64 long descriptor。

short-descriptor 格式的 L1 table 有 4096 個 4-byte entries，總大小為 $4096 \times 4 = 16,384$ bytes，即 16 KiB。`TTBCR.N=0` 時所有輸入 VA 都由 TTBR0 選 root，並以 VA `[31:20]` 索引完整 L1 table；TTBR0 的 table base 因此必須 16 KiB 對齊。

$$L1_address = TTBR0 + ((VA \gg 20) \& 0xFFF) \times 4.$$

coarse L1 page-table descriptor 指向 L2 table。short-descriptor 的 L2 table 有 256 個 4-byte entries，總大小 1 KiB，base 只需 1 KiB 對齊；硬體使用 descriptor 的 `[31:10]` 作為 table base，因此不能任意插入非對齊位址。

$$L2_address = L2_base + ((VA \gg 12) \& 0xFF) \times 4, \qquad PA = frame + (VA \& 0xFFF).$$

8.8　ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

本例的四個 table 區間完整且互不重疊：

storage	base	range	size	alignment check
P1 root	`0x8000`	`0x8000–0xBFFF`	16 KiB	`0x8000 % 0x4000=0`
P2 root	`0xC000`	`0xC000–0xFFFF`	16 KiB	`0xC000 % 0x4000=0`
P1 L2	`0x10000`	`0x10000–0x103FF`	1 KiB	`0x10000 % 0x400=0`
P2 L2	`0x10400`	`0x10400–0x107FF`	1 KiB	`0x10400 % 0x400=0`

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.8 ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

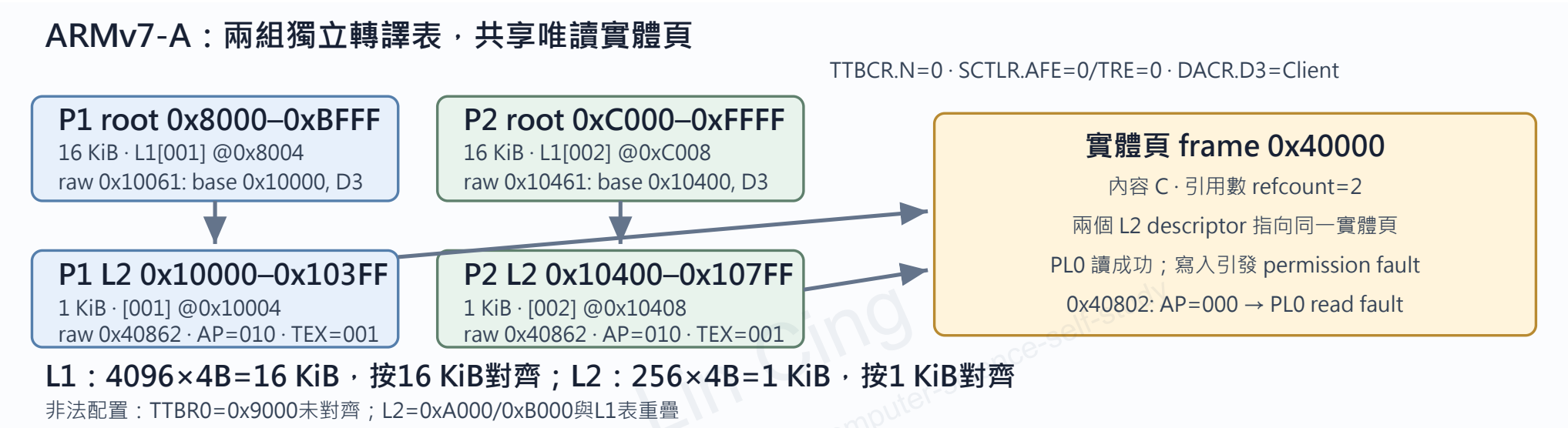
P1 VA `0x00101000` 與 P2 VA `0x00202000` 的實際位址為：

process	VA	L1 index → entry address	L2 base	L2 index → entry address	frame	PA
P1	`0x00101000`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40000`
P2	`0x00202000`	`0x002→0xC008`	`0x10400`	`0x002→0x10408`	`0x40000`	`0x40000`
P1，非零 offset	`0x00101A5C`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40A5C`

第三列的 page offset 是 VA `[11:0]=0xA5C`；它不參與任一層索引，會完整保留至輸出位址。只保留最低 8 位會錯把 PA 算成 `0x4005C`。

每個 address space 都保留完整 root 儲存。即使 VA 只使用少數 entries，也不能只保存那幾筆 descriptor；其餘 entries 必須有架構允許的 invalid 或其他有效內容。兩張 16 KiB roots 與兩張 1 KiB L2 tables 佔用不同的 physical ranges。

8.8 ARMv7-A AArch32：short-descriptor 的兩層 walk （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.8　raw descriptors、domain、AFE 與 AP

本例在每個 L1 page-table descriptor 使用 domain 3，並設定 `DACR.D3=0b01`，即 Client。domain 只存在 L1 page-table descriptor；L2 entry 繼承該 domain。

當 `SCTLR.AFE=0` 時，使用 full AP[2:0] 模型。small page 的 AP[2] 在 bit9、AP[1:0] 在 bits[5:4]；不能套用 section descriptor 的 bits[11:10]。本例 L2 解碼後 `AP[2:0]=010`：PL1 read/write、PL0 read-only，所以 PL0 write 產生 Permission fault。

可從 raw word 直接解碼：

descriptor	raw word	解碼
P1 L1	`0x10061`	bits[1:0]=01 coarse；table base=`0x10000`；domain=3
P2 L1	`0x10461`	bits[1:0]=01 coarse；table base=`0x10400`；domain=3
P1/P2 L2	`0x40862`	frame=`0x40000`；type bits[1:0]=10；XN=0；nG=1；AP[2:0]=010；TEX=001、C=B=0

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.8 raw descriptors、domain、AFE 與 AP （續）

small-page descriptor 的 `field = (raw_word >> shift) & mask` 必須逐項套用：`AP2=(raw>>9)&1`、`AP1=(raw>>5)&1`、`AP0=(raw>>4)&1`、`TEX=(raw>>6)&7`、`C=(raw>>3)&1`、`B=(raw>>2)&1`、`XN=raw&1`。本例得到 AP010、TEX001、C=B=0、XN=0、nG=1。

`TEX=001,C=B=0` 在 `SCTLR.TRE=0` 下解為 Normal non-cacheable。instruction fetch 不能只由 XN=0 推出；本模型另固定 `SCTLR.WXN=UWXN=0`、mapping 可讀、domain Client，且沒有其他實作限制。若只改記憶體屬性為 strongly ordered，不能憑 XN=0 宣稱一定可直接執行。

同一個 frame 的 descriptor 可以給出不同權限。比較下列兩個設定：

raw word	正確解碼	結果
`0x40802`	AP[2:0]=000；TEX=0,C=B=0	AP 禁止 PL0 read/write；frame 位址有效也無法通過 read 權限檢查
`0x40022`	AP[2:0]=010；TEX=0,C=B=0	只示範 data AP；Strongly-ordered 不可藉 XN=0 推 instruction fetch

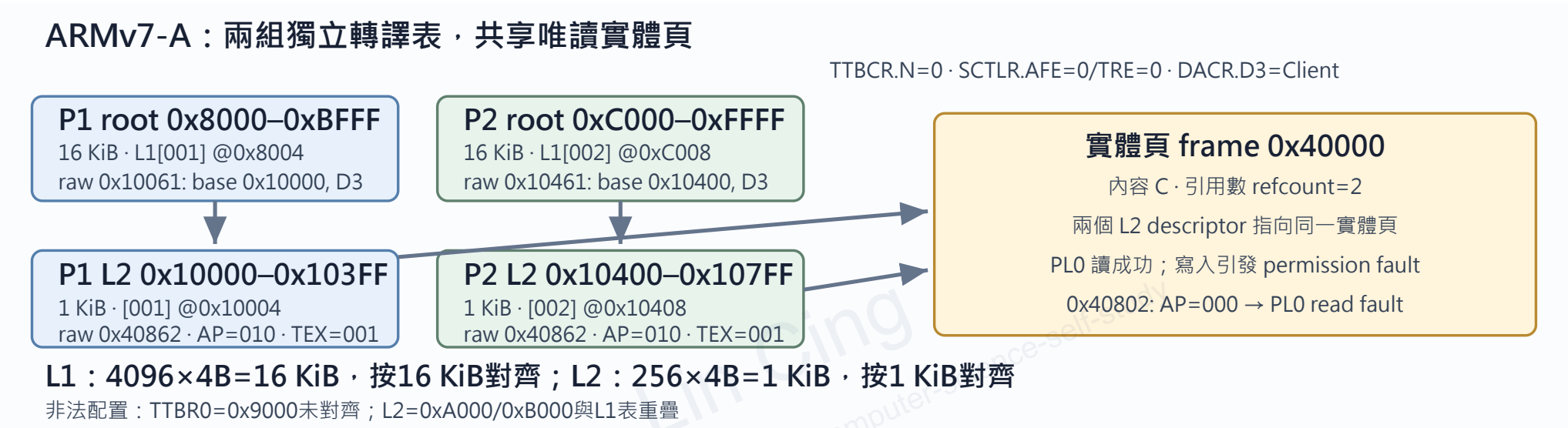
解碼時必須分別讀取 type、frame、AP 與記憶體屬性；只認出 frame 位址，尚不足以判定存取結果。

8.8 raw descriptors、domain、AFE 與 AP （續）

permission 不是只看 L2 word。DACR D3=Client 時才檢查 AP；若 D3=Manager（`0b11`），short-descriptor 的 AP 不被檢查，因此不能再用相同 L2 word 推出 write 必 fault。若 `SCTLR.AFE=1`，AP[0] 被重新定義成 Access flag，AP 模型改為 AP[2:1]；必須重新解碼 descriptor，不能沿用 AFE=0 的結論。

條件	PL0 read	PL0 write	理由
AFE=0、D3=Client、AP=010	成功	Permission fault	AP 對 PL0 授予 read、不授予 write
AFE=0、D3=No access	Domain fault	Domain fault	AP 檢查尚未成為成功證據
AFE=0、D3=Manager	不因 AP 被拒	不因 AP 被拒	Manager 不檢查 AP；其他架構 fault 另計
AFE=1、Client、同一字 `0x40862`	Access flag fault	Access flag fault	AP[0]=0；本例採軟體設定 Access flag
AFE=1、Client、改成 `0x40872`	成功	成功	AP[0]=1，AP[2:1]=01；此模型給 PL0 read/write，與 AFE=0 的 AP010 不同

8.8 raw descriptors、domain、AFE 與 AP （續）



Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.8　權限 trace、mapping 移除與 TLB

初始 frame `0x40000` 的 reference count=2；兩張 L2 entries 都是 `0x40862`，其 parent domains 都是 Client。TLB 起初沒有這個 translation。

狀態與操作	translation	domain/AP	結果	frame 或 refcount
S0：P1 read	`0x00101000→0x40000`	Client；AP=010 允許 PL0 read	success	C；refcount 2
S0：P1 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變
S0：P2 read	`0x00202000→0x40000`	Client；AP=010 允許 PL0 read	success	C；refcount 2
S0：P2 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.8　權限 trace、mapping 移除與 TLB （續）

狀態與操作	translation	domain/AP	結果	frame 或 refcount
S1：P1 descriptor absent	P1 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	P2 仍映射；refcount 1
S2：P2 read after P1 removal	獨立 P2 tables 仍有效	Client/read allowed	success	refcount 1
S3：P2 descriptor absent	P2 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	refcount 0，所有舊 translation 已失效後 frame 才可回收

translation fault 與 permission fault 是不同證據。S1 的 P1 沒有效 mapping，因此不能說「頁存在只是不能寫」；S0 的 write 已有有效 mapping，且在 Client domain 下 AP 拒絕 PL0 write。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

8.8　權限 trace、mapping 移除與 TLB （續）

更新 descriptor 後，要依架構規則完成對應 VA／ASID 的 TLB maintenance 與必要 barrier。本例 nG=1，translation 屬於各自的 ASID；ASID 不決定 domain 或 AP。stale TLB entry 可能保留舊 translation 或權限，因此上表的 unmap 結果以失效操作完成為前提，不能在清除 PTE 後立即回收仍可能被舊 translation 存取的 frame。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.8　先驗 storage layout，再談共享 frame

下列配置即使能算出 entry 位址，仍不構成合法硬體配置：

錯誤配置	問題	判定
P2 TTBR0=`0x9000`，N=0	`0x9000%0x4000=0x1000`，未達 16 KiB 對齊	invalid root
P1 root=`0x8000`、P2 root=`0x9000`	ranges `0x8000–0xBFFF` 與 `0x9000–0xCFFF` 重疊	invalid ownership
P1 L2=`0xA000`	落在 P1 root `0x8000–0xBFFF` 內	table storage 重疊
P2 L2=`0xB000`	同樣落在 P1 root 內	table storage 重疊
只為 16 KiB L1 配置 4 KiB	4096 個 4-byte entries需要完整 16 KiB	storage 不足

修正後，P1/P2 各有獨立 roots 與 L2 tables，只有最後的 `frame 0x40000` 被共享。page identity、各層 storage、domain、AP、XN、frame reference count 與 TLB state 必須一起核對。

8.8 先驗 storage layout，再談共享 frame （續）

若 P1 descriptor 真的可寫，P1 的 write 會修改 P2 看見的共享 frame；這是 shared writable page 的語意，不是 read-only code。若系統要讓 P1 修改而 P2 保留舊內容，應使用 copy-on-write 或新 frame，不能只改其中一份 descriptor。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.9 總結

- segmentation 與 paging 解不同限制：前者貼近邏輯結構，後者簡化配置；兩者可共同提供隔離與共享。
- TLB、page table 與 page fault handler 構成完整位址轉換路徑。任何一致性更新都必須讓舊翻譯不可再被命中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.9　模型、時序與反例：總結

模型假設。 logical 到 physical 的每個 mapping 都由硬體驗證，fault handler 的恢復點精確。

不變量與時序。 valid、permission 與 mapping 三者一致；TLB/page table 與 program-visible memory 沒有舊狀態殘留。

反例或邊界。 只更新 page frame 不更新 offset 算式，後續位址會偏移。

完整算例。 固定三事件：VA 超 VMA → invalid-address fault；合法 mapped PTE 但 write=0，執行 write → protection fault；TLB miss 但 table entry valid → page-table walk，不報 fault。三者恢復結果分別是終止、拒絕該 operation、完成 translation。

思考題。 哪些錯誤可由 page fault 恢復，哪些必須終止請求？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章收束

- page mapping 與權限是同一個 entry 契約的兩部分
- internal/external fragmentation 必須分類
- EMAT 是模型，需列出 cache 與 fault 假設

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

練習與參考資料

練習題與詳解分別見本章獨立 PDF。

- Operating System Concepts, 9th Edition，第 8 章，印刷頁 351-394。
- *Operating System Concepts*, 9th Edition, Sections 8.7-8.8, pp. 383-389：IA-32 segmentation/paging、10/10/12 與 4 MiB page、x86-64 48-bit VA 與四層 table、ARM short-descriptor 的 4 KiB/16 KiB pages 與 1 MiB/16 MiB sections。
- Intel 64 and IA-32 Architectures Software Developer's Manual：paging-structure entry addresses、canonical address、large-page termination，以及 U/S、R/W、NX 的 effective permission。
- Armv7-A/R Architecture Reference Manual, ARM DDI 0406C.d：B3.5 定義 two-level short descriptors、1 KiB L2 table 與 small pages；B3-1327 Table B3-1 規定 `TTBCR.N=0` 時 TTBR0 root 為 16 KiB、VA `[31:20]` 索引；B3-1329 的 TTBR0 base 為 `[31:14-N]`，N=0 即 16 KiB 對齊；B3-1354、B3-1358 與 B4-1554 規定 AFE、AP、Client/Manager 與 DACR；B4-1717 固定 TTBCR.N 欄位。

練習與參考資料 （續）

- Linux v3.2 ARM page-table helper：部分權限與 Linux-table 表示的實作對照；硬體 short-descriptor 的 type、AP、domain 與 cache 欄位以 ARM 架構手冊為準。
- 本章位址與權限 trace 固定 execution mode、table base、entry size、page size 與 entry values。x86-64 的 `0x1000–0x4000` 是教學配置；ARM 的 roots `0x8000`/`0xC000` 各佔 16 KiB，L2 tables `0x10000`/`0x10400` 各佔 1 KiB，四區互不重疊。這些 physical bases 不是任何機器的固定位址。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study